Claude

Etiqueta de modelo: Opus 5.5 Alto
Leida: 2026-11-09T21:22:50.456Z
Caracteres: 8245
Fuentes citadas (verificables): 44
Enlaces en el DOM: 44
Fin de respuesta: observado

Pregunta

Contexto: ChatCouncil es una app de investigación que abre diez páginas web reales de proveedores de IA a la vez, cada una en su propio webview nativo con la sesión del usuario; escribe en sus campos de texto, lee sus respuestas del DOM, adjunta archivos y arma un informe. Se está reescribiendo con el núcleo en Rust y un solo código para Windows, macOS, Linux, Android e iOS (escritorio, móvil y tableta), con el rendimiento (RAM, CPU, tiempo de arranque y tamaño) como prioridad absoluta.

Ya medido en Windows, con diez webviews nativos de WebView2 en la misma máquina: Flutter con plugins que dibujan por textura gasta unos 5 puntos de CPU extra cuando las páginas redibujan; Flutter con ventanas nativas y Slint con ventanas nativas (wry) gastan casi cero en el proceso propio; Slint arranca en 0,76 s contra 3,35 s de Flutter, pesa 11 MiB contra 25,5 MiB y usa 74 MB de RAM en el proceso propio contra 122 MB. Se descartaron Dioxus (su interfaz sería un webview más) y Rune (no tiene Android ni iOS).

Pregunta: ¿qué opciones de interfaz y de manejo de varios webviews nativos simultáneos, viables hoy para esas cinco plataformas con un solo código y un núcleo en Rust, no se están considerando, y cómo se comparan en rendimiento medido con Slint y con Flutter? Para cada opción: fuentes verificables con fecha, estado real en cada plataforma (en especial varios webviews nativos simultáneos en Android e iOS), madurez, licencia y riesgos.

Respuesta

Claude respondió: Voy a investigar las alternativas no consideradas (Tauri/wry en móvil, Qt + cxx-qt, Avalonia/Uno, Compose Multiplatform, Makepad/Dioxus Native, núcleo Rust con…
Revisando las limitaciones de webviews en iOS☐
Revisando las limitaciones de webviews en iOS
Voy a investigar las alternativas no consideradas (Tauri/wry en móvil, Qt + cxx-qt, Avalonia/Uno, Compose Multiplatform, Makepad/Dioxus Native, núcleo Rust con capas nativas) con foco en varios webviews simultáneos en Android e iOS, y luego contrastar con mediciones publicadas.

Buscó en la web, leyó 7 páginas□
Buscó en la web, leyó 7 páginas
Hallazgo principal: lo que mediste con Slint + wry en Windows no se puede llevar tal cual a Android ni a iOS. La versión actual de wry no puede ubicar varios webviews dentro de una misma pantalla en el móvil, así que hay que escribir esa capa aparte o usar un framework que ya la tenga.
Qué pregunta respondo
Qué opciones viables hoy (9-nov-2026), con un solo código y núcleo en Rust para Windows, macOS, Linux, Android e iOS, no estás considerando, y cómo rinden frente a tus mediciones: Slint 74 MB, 0,76 s y 11 MiB; Flutter 122 MB, 3,35 s y 25,5 MiB.
Resumen

wry no ubica varios webviews en el móvil (confianza alta, lo vi en el código de wry 0.57.1):

Android: el código tiene un solo espacio de webview por Activity, y al crear uno lo pone como contenido completo de la Activity. Además, set_bounds no hace nada. docs□docs□
iOS: el webview hijo se agrega como subvista, pero toma el tamaño de la vista padre y no los límites pedidos; set_bounds solo funciona en macOS. docs.rs□
Integración con Slint: wry pide que la Activity principal herede de AppCompatActivity, idealmente la WryActivity generada. Slint en Android usa winit. Un usuario de winit con GameActivity contó que no logró hacer funcionar wry en Android (es un comentario de foro: solo prueba que alguien lo dijo). docs□dependabot□

Tauri tampoco lo resuelve en el móvil. Su soporte móvil de marzo de 2026 permite varias ventanas independientes a pantalla completa, cada una con su webview, pero no permite poner sus vistas dentro de una jerarquía de vistas nativa. Además, su interfaz sería otro webview, el mismo motivo por el que descartaste Dioxus. github□github□
La opción no considerada más completa es Avalonia (.NET) con su control NativeWebView. Su tabla de soporte marca NativeWebView disponible en Windows, macOS, Linux, iOS y Android. El precio es cargar el runtime de .NET y conectar tu núcleo Rust por FFI. avaloniaui□
Qt WebView (Qt 6.12) con CXX-Qt usa el webview del sistema en cuatro plataformas, pero en Linux depende de Qt WebEngine, es decir, Chromium empaquetado. Eso va contra tu prioridad de tamaño en Linux. qt□
No encontré ninguna medición pública comparable con 10 webviews ni contra Slint o Flutter. Las cifras que existen son del propio fabricante o de apps distintas. Para comparar de verdad hay que medir con tu propio arnés.

Método y límites
Hice unas 20 búsquedas y lecturas: código fuente de wry en docs.rs, documentación oficial (Qt, Avalonia, Uno, Android/Chromium, Apple), notas de versión y blogs con mediciones. No ejecuté nada. «Observado» significa que lo leí en código o en documentación; «declarado por el fabricante» significa que no es una medición independiente.
Límites del sistema operativo (afectan a cualquier opción)

Android: las apps tienen un solo proceso de renderizado para todos sus WebViews, salvo casos raros. Si ese proceso termina, cada WebView que lo comparte debe manejar onRenderProcessGone; si alguno no lo hace, también muere la app. Una salida es usar perfiles: cada perfil de WebView tiene su propio proceso de renderizado. WebView and memory +2□
iOS: WebKit le da a cada webview su propio proceso hasta un límite que no publica. Esos procesos se pueden matar sin afectar al proceso de la app, así que hay que reponer cada página que se cierre. Diez páginas pesadas en un iPhone probablemente llevan a cierres por falta de memoria; esto es inferencia, no lo medí. apple□embrace□

Opciones no consideradas
Avalonia + NativeWebView (C#)

Android/iOS: las vistas nativas se insertan con NativeControlHost, pero siempre quedan encima del contenido de Avalonia; no se puede poner un control de Avalonia sobre una vista nativa. avaloniaui□
Linux: usa WPE WebKit si está instalado y WebKitGTK si no. avaloniaui□
Licencia: el paquete oficial requiere licencia Avalonia Accelerate, que es de pago. Existe una alternativa comunitaria con licencia MIT, que hace pocos meses estaba en 0.1.0-alpha.1 (10/3/2026). Avalonia.Controls.WebView 11.3.15 +2□
Rendimiento: no encontré mediciones.
Riesgos: el runtime de .NET y el FFI hacia Rust; que funcionen varias instancias a la vez en móvil está sin verificar.

Uno Platform (C#)

Webview: el fabricante afirma que WebView2 funciona en todas sus plataformas. Licencia: Apache 2.0. platform□infoq□
Rendimiento (declarado por el fabricante, app distinta a la tuya): en Windows con compilación nativa anticipada, 300 ms al primer cuadro y 212 MB; en Android, 654 ms y 301 MB. Esa cifra de RAM es peor que la de Flutter que mediste. platform□

Qt 6.12 + Qt WebView + CXX-Qt (o Qt Bridge for Rust)

Motores: WebView2 en Windows, WKWebView en macOS e iOS, Android WebView en Android. qt□
Licencia: comercial, LGPLv3 o GPLv2. CXX-Qt es MIT o Apache-2.0. qt□github□
Riesgos:

Sigue abierta la petición de poder poner elementos de interfaz encima del WebView nativo en iOS y Android. qt□
Qt Bridge for Rust salió como versión preliminar el 27-oct-2026, pero no publica lista de plataformas ni menciona Android o iOS. qt□
En Linux arrastra Chromium completo.

Rendimiento: no encontré mediciones fiables.

Compose Multiplatform (Kotlin)

Madurez: iOS estable desde la versión 1.8.0 (mayo de 2025). jetbrains☐
Webview: en escritorio Compose no trae webview propio; los webviews nativos del escritorio no están disponibles en Compose Multiplatform. Dependerías de librerías de terceros. teamdev☐
Rendimiento: el escritorio corre sobre la JVM, lo que probablemente penaliza RAM y arranque frente a Slint (inferencia, no lo medí).

Núcleo Rust + interfaz nativa por plataforma (Crux)

Crux comparte la lógica escrita una vez con interfaz nativa en cada plataforma. red-badger☐
Probablemente es lo de mejor rendimiento posible, pero rompe tu requisito de un solo código de interfaz.

Descartadas por cobertura de plataformas

Lynx: soporta oficialmente macOS y Windows, sin Linux. lynxjs☐
Dioxus Native (Blitz): el soporte en iOS y Android es experimental, y no encontré webview embebido. mintlify☐

Conflictos y vacíos

La medición Tauri contra Electron (142 MB contra 149 MB) llegó citada por un medio; no pude abrir la fuente original. tech-insider☐
No verifiqué si NativeWebView, Uno o Qt aguantan 10 instancias a la vez en móvil.
No verifiqué el estado de Makepad para webviews.

Siguiente paso recomendado
Antes de elegir interfaz, haz un prototipo solo de la parte móvil. Debe ser una capa mínima en Kotlin y Swift, llamada desde Rust, que:

cree N WebViews con su posición y tamaño;
ejecute JavaScript para escribir y leer del DOM;
adjunte archivos.

Mídelo con 10 proveedores reales en un Android de gama baja y en un iPhone.

Qué confirmaría el plan: que funcione con los 10 sin cierres del proceso de renderizado. En ese caso, Slint con esa capa propia sigue siendo la base más liviana.
Qué lo descartaría: cierres por falta de memoria en iOS, o que la Activity propia no conviva con Slint en Android.
Costo si sale mal: semanas de trabajo en la capa nativa.
Plan B: medir Avalonia + NativeWebView con el mismo arnés que usaste para Slint y Flutter.

Fuentes
Fuente | Tipo | Fecha | wry android/mod.rs, main_pipe.rs, wkwebview/mod.rs | Primaria (código) |

wry 0.57.1 |
Tauri PR #14484 | Primaria | 23-mar-2026 |
Android: WebView y memoria, Chromium renderer | Primaria | s/f |
Apple WKProcessPool | Primaria | s/f |
Avalonia WebView, vistas nativas en Android | Primaria | 2026 |
Qt WebView (blog), Qt Bridges | Primaria | 4-may / 27-oct-2026 |
Uno: compilación nativa anticipada | Medición del fabricante | 6-ago-2026 |
JetBrains: Compose 1.8.0 | Primaria | may-2025 |
Lynx 3.7 | Primaria | 16-abr-2026 |
tech-insider (Dawloom) | Prensa | oct-2026 |

Texto capturado tal cual de la interfaz del proveedor, sin corregir ni recortar.
Este documento es la respuesta a la pregunta, no la evaluacion que este
proveedor hizo de las respuestas de los demas.